

An Explainable Multi-Agent Framework for DeFi Price Manipulation Security Auditing

ZHENG Bowen (Student ID: 21286853)

1 Background

Decentralized Finance (DeFi) [1] has introduced a wide range of financial applications built on blockchain infrastructure, including decentralized exchanges, lending platforms, derivatives protocols, and yield optimization systems. Unlike traditional financial systems, DeFi protocols execute financial logic through publicly accessible smart contracts, allowing users to interact with automated mechanisms without centralized intermediaries. However, the permissionless nature of DeFi systems also introduce significant security risks. Many high-impact DeFi exploits are not caused by conventional programming errors alone, but by interactions between economic mechanisms, market states, transaction ordering, and dependencies between multiple contracts. Price manipulation attacks are among the most significant categories of such vulnerabilities, where attackers exploit weaknesses in oracle mechanisms, liquidity structures, or protocol assumptions to influence asset prices and extract economic benefits (e.g., [2]). Traditional smart contract auditing approaches mainly rely on manually written security rules, symbolic analysis, and expert review [3, 4]. While these methods are effective for identifying predefined coding issues, they often struggle with complex economic vulnerabilities that require understanding of protocol behavior and attack strategies. Recent advances in large language models (LLMs) have demonstrated strong capability in code understanding and reasoning [5, 6], providing new possibilities for automated security analysis. However, directly applying LLMs to security auditing remains challenging due to hallucination risks, inconsistent reasoning processes, and difficulty in maintaining reliable evidence chains. Therefore, automated DeFi security auditing requires a framework that combines structured analysis with semantic reasoning. Such a system should not only identify potential vulnerabilities, but also understand protocol context, reconstruct possible attack paths, evaluate economic feasibility, and provide interpretable confidence assessments.

2 Motivation

The limitations of existing auditing approaches indicate that automated DeFi security analysis requires more than vulnerability pattern matching. Price manipulation attacks usually emerge from interactions among multiple components involved in financial mechanisms, including price sources, liquidity states, transaction execution conditions [7], privileged operations, and external protocol dependencies. Consequently, effective analysis requires multiple forms of reasoning, such as protocol understanding, vulnerability identification, attack reconstruction, and economic assessment. A single general-purpose auditing model is insufficient for such a complex task[8]. Although LLMS provide strong semantic understanding capabilities, applying them directly to the entire auditing workflow may introduce unreliable conclusions due to uncertain reasoning processes and insufficient control over analysis scope. On the other hand, many security analysis tasks involve structured decisions that can be handled more reliably using deterministic approaches, such as protocol classification, vulnerability filtering, and quantitative calculations. These observations motivate a hybrid auditing framework that combines deterministic analysis with language-model-based reasoning. Deterministic components are responsible for tasks with clear rules, while LLM-based agents focus on contextual reasoning and complex security analysis. The positioning of different automated auditing approaches is summarized in Table 1

Table 1: Comparison of automated auditing approaches

Approach	Main Capability	Main Limitation
Rule-based security analyzers	Reliable detection of predefined vulnerability patterns and coding issues.	Lack knowledge of economic mechanisms and complex protocol interactions.
Single LLM-based auditing	Strong semantic interpretation of contract logic and natural-language reasoning.	Vulnerable to hallucination, inconsistent analysis, and insufficient verification.
General Multi-agent framework	Specialized reasoning across different analytical stages through coordinated agents.	Requires explicit workflow design and information management.
Proposed hybrid framework	Hybrid deterministic analysis combined with multi-agent reasoning and confidence calibration.	Currently focused on DeFi price manipulation scenarios.

3 Technical Approach

The proposed system is designed as an explainable multi-agent auditing framework coordinated by an Audit Orchestrator. Instead of treating vulnerability analysis as a single end-to-end reasoning problem, the workflow is divided

into several specialized stages, including protocol identification, context construction, vulnerability analysis, attack reconstruction, cost estimation, confidence calibration, and report generation. Each stage produces structured intermediate results that are passed to subsequent stages. This separation allows individual agents to maintain clearly defined responsibilities and avoids uncontrolled information exchange between components. The orchestrator manages the execution order, reasoning flow, and failure handling mechanisms throughout the auditing process. A central principle of the architecture is the separation between deterministic computation and semantic reasoning. Tasks with explicit rules, such as protocol classification, quantitative cost calculation, and confidence aggregation, are handled through deterministic mechanisms. LLMs are reserved for tasks requiring contextual understanding, including vulnerability interpretation, attack reasoning, and explanation generation. External dependencies, including blockchain data sources and LLMs, are considered potentially unreliable components. Therefore, the system incorporates fallback mechanisms that allow the audit process to continue when individual components become unavailable.

3.1 Vulnerability Pattern Taxonomy

The vulnerability taxonomy is designed based on the observation that DeFi price manipulation attacks are generally caused by interactions between multiple components involved in price formation rather than isolated programming mistakes. These components include oracle mechanisms, liquidity structures, transaction execution conditions, privileged operations, mathematical calculations, and dependencies between protocols. Therefore, the system organizes the 21 vulnerability patterns into six categories: Oracle Dependency, Liquidity and Reserve Manipulation, Transaction Ordering, Access Control Risks, Calculation Logic Bugs, and Composability Risks. This mechanism-oriented classification is consistent with established smart contract security taxonomies, such as the Smart Contract Weakness Classification (SWC) Registry [9] and vulnerability categories commonly used in security auditing practice, while extending the focus toward DeFi-specific economic manipulation scenarios. Compared with classifications based only on code-level weaknesses, this taxonomy emphasizes how attackers influence protocol price formation and provides a structured basis for vulnerability prioritization, reasoning, and attack reconstruction.

The vulnerability pattern classification is defined in Table 2; subsequent discussions refer to patterns by their IDs.

Table 2: Vulnerability patterns

Category	Pattern ID	Pattern Name	Key Code Feature
Liquidity / Reserve Manip.	LR-01	Flash Loan Reserve Manipulation	Borrow large liquidity in same tx to affect AMM price.
	LR-02	Large-Trade Slippage Manipulation	Large swap manipulates average price without effective slippage protection.
	LR-03	Virtual Liquidity Manipulation	Donate or remove liquidity to alter pool state.
Oracle Manipulation	OD-01	Single DEX as Price Source	Direct <code>getReserves()</code> call for pricing.
	OD-02	Short TWAP Window	Time-weighted window too short to resist flash manipulation.
	OD-03	LP Price Not Excluding Self-Holding	Price calculation does not exclude protocol’s own holdings.
	OD-04	Cross-Chain Bridge Price Unverified	Relies on cross-chain price messages without cross-validation.
	OD-05	Stablecoin Depeg Ignored	Protocol still accepts stablecoin at peg despite deviation.
Tx Ordering Manip.	TO-01	Order Dependency	Critical computation depends on tx order; MEV can front-run.
	TO-02	No Slippage Protection	Swap lacks user-specified minimum output check.
	TO-03	Sandwich Attack	Attacker can trade before and after user to squeeze profit.
Access Control Risks	AC-01	Oracle Address Unilaterally Updatable	Privileged function lacks timelock or multi-sig.
	AC-02	Privileged Econ. Parameter Adjustment	Fees, collateral ratios adjustable without constraints.
	AC-03	Mint/Burn Authority Concentrated	Unlimited mint/burn can affect price.
Calc. Logic Bugs	CL-01	Exploitable Rounding Direction	Division/modulo remainder repeatedly extractable.
	CL-02	Decimal Mismatch	Tokens with different decimals used without normalization.
	CL-03	Inappropriate AMM Curve/Params	Custom curve or extreme volumes unprotected; price deviation.

Table 2: Vulnerability patterns (continued)

Category	Pattern ID	Pattern Name	Key Code Feature
Composability Risks	CR-01	External Protocol as Sole Price Source	Core logic calls external DEX/bridge without fallback.
	CR-02	Unverified External LP Value Assumption	LP tokens as collateral without verifying underlyings.
	CR-03	Cross-Protocol Call Return Not Checked	Return value not validated; revert not handled.
	CR-04	Cross-Protocol Price Dependency Unverified	Manipulating A’s liquidity affects B’s pricing without safeguards.

3.2 Audit Orchestrator and Agent Pipeline

The orchestrator is the single entry point that launches the analysis pipeline and dynamically adjusts reasoning depth based on early findings. The entire workflow is divided into seven sequential stages, each producing an immutable structured data object that becomes the input for the next stage.

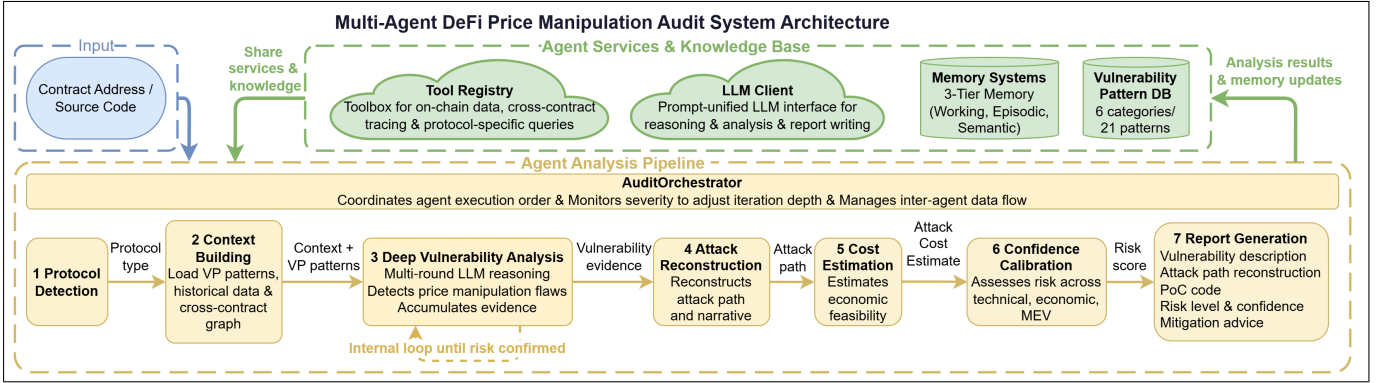


Figure 1: System architecture overview.

Stage 1: Protocol Identification. The first stage establishes an initial understanding of the target protocol before deeper vulnerability analysis begins. Since protocol classification mainly depends on recognizable structural characteristics, such as contract interfaces, function names, and state variables, this stage adopts deterministic rule-based analysis instead of LLM inference. The detector classifies contracts into nine DeFi categories, including DEX/AMM, lending, perpetuals, yield aggregator, cross-chain bridge, stablecoin, options, liquid staking, and unknown categories [1]. The resulting protocol profile determines a prioritized subset of vulnerability patterns from the taxonomy, allowing later analysis to focus on risks that are more relevant to the target protocol.

Stage 2: Context Building. After protocol identification, the system constructs an analysis context containing relevant vulnerability patterns, historical cases, and potential external dependencies. Rather than providing the entire vulnerability knowledge base to the reasoning agent, the context is selectively filtered according to the identified protocol characteristics. The memory system maintains three types of information with different purposes. Working memory stores temporary reasoning states within an audit session, episodic memory preserves previous audit experiences, and semantic memory supports retrieval of related vulnerability knowledge. When necessary, cross-contract analysis is performed to identify external dependencies that may influence price formation. Historical retrieval and external analysis are treated as supplementary information sources rather than mandatory requirements. If additional data sources are unavailable, the core auditing process can continue using the available contract information.

Stage 3: Deep Vulnerability Analysis. This stage represents the primary reasoning component of the system and adopts an observe-think-act-update workflow for iterative vulnerability investigation. Instead of producing a single conclusion from initial input, the agent repeatedly examines potential risks, refines previous findings, and determines whether further analysis is required. The reasoning process is guided by the vulnerability patterns selected during earlier stages. This prevents unnecessary exploration of unrelated security issues and focuses the analysis on price manipulation risks. When evidence remains insufficient, additional reasoning iterations are performed to examine different code components or attack mechanisms. To reduce unreliable vulnerability reports, deterministic verification is applied to several commonly misidentified patterns. These checks examine whether relevant protection mechanisms actually exist before accepting or suppressing potential findings.

Stage 4: Attack Reconstruction. For each confirmed vulnerability, the system generates a structured reconstruction of how an attacker could exploit the identified weakness. Rather than producing independent descriptions for every vulnerability instance, the reconstruction process follows a layered approach that combines general attack mechanisms

with contract-specific information. The first layer defines common attack structures for major vulnerability categories, including preparation, execution, manipulation, exploitation, profit extraction, and cleanup stages. The second layer incorporates characteristics specific to individual vulnerability patterns, allowing different attack mechanisms to preserve their unique properties. Finally, contract-level details identified during analysis are integrated into the reconstruction. This hierarchical strategy improves consistency across similar attack scenarios while maintaining sufficient flexibility for protocol-specific analysis. The generated reconstruction also provides an interpretable connection between vulnerability identification and potential economic impact.

Stage 5: Cost Estimation. The system estimates the economic feasibility of potential attacks using deterministic calculations based on external market information and attack-specific execution requirements. The estimation considers factors such as transaction costs, asset prices, flash-loan fees, and additional expenses introduced by different attack patterns. Unlike vulnerability reasoning, which requires semantic interpretation, cost estimation is treated as a quantitative analysis problem. Therefore, the system avoids relying on LLM judgment for numerical evaluation and preserves the underlying calculation information to improve reproducibility.

Stage 6: Confidence Calibration. Security auditing requires not only identifying potential vulnerabilities but also determining the reliability of each finding. This requirement becomes particularly important in LLM-assisted auditing, where generated explanations may appear reasonable while still containing incomplete evidence or uncertain assumptions. To address this issue, the system introduces an independent confidence calibration stage after vulnerability analysis[15]. Instead of directly accepting language-model-generated findings, each vulnerability report is evaluated through multiple reliability dimensions. The calibration model considers five factors, summarized in Table 3.

Table 3: Confidence calibration dimensions

Dimension	Description
Pattern-Match Priority	Evaluates whether the detected vulnerability corresponds to a highly relevant pattern for the identified protocol type.
Economic Feasibility	Evaluates whether the potential attack yields meaningful economic impact under realistic conditions.
Source Code Availability	Measures whether sufficient implementation-level evidence is available.
Historical Case Support	Indicates whether similar vulnerabilities have been observed in past DeFi incidents.
Cross-Round Consistency	Measures whether the finding remains stable across multiple reasoning iterations.

Each dimension is evaluated independently and contributes to the final confidence score through weighted aggregation. The individual factors are preserved together with the final score, allowing reviewers to understand why a vulnerability receives a specific confidence level. This mechanism separates vulnerability discovery from vulnerability verification. Language-model reasoning is used to explore potential risks, while confidence calibration provides an additional assessment layer before findings are included in the final audit report.

Stage 7: Report Generation. After all analytical stages are completed, the orchestrator combines structured outputs from previous agents into the final audit report. The LLM is primarily used for organizing findings and improving explanation quality, while key analytical results, including vulnerability categories, confidence assessments, attack costs, and supporting evidence, remain derived from previous stages. This separation prevents the reporting stage from modifying security conclusions and ensures that the final report remains consistent with the underlying analysis process.

3.3 Learning and Evolution Mechanism

The system incorporates a knowledge accumulation mechanism that allows previous audit experiences to support future vulnerability analysis. Unlike model fine-tuning approaches, the mechanism does not modify the underlying LLM. Instead, improvements are achieved through storing and retrieving validated audit knowledge. Before entering the knowledge base, audit findings pass through validation criteria to reduce the risk of propagating unreliable information. High-confidence findings supported by sufficient evidence are retained, while uncertain observations are excluded to prevent inaccurate experiences from influencing future analysis. The accumulated knowledge is organized into different memory layers. Episodic memory stores complete historical audit cases, including vulnerability descriptions and analysis outcomes, while semantic memory preserves generalized vulnerability-related information for similarity-based retrieval. During future audits, retrieved experiences provide additional context rather than replacing direct analysis of the target contract. To maintain experimental reliability, knowledge accumulation is disabled during benchmark evaluation. Therefore, measured performance reflects the inherent capability of the auditing framework instead of improvements caused by previous test cases.

3.4 Cross-Contract Tracing

Many DeFi price manipulation attacks involve multiple interacting contracts, making single-contract analysis insufficient for identifying hidden dependencies. External calls, however, are often represented only as contract addresses, which provide limited semantic information for vulnerability reasoning. To address this issue, the system performs cross-contract tracing by extracting external call relationships and constructing a bounded call graph. Known protocol

addresses are mapped to semantic identities, and relevant interactions are annotated with corresponding vulnerability patterns. For example, price-related external calls can be associated with oracle dependency risks, allowing the reasoning agent to recognize potential attack surfaces across protocol boundaries. The tracing mechanism is designed as an auxiliary analysis component rather than a mandatory dependency. Dynamic calls that cannot be fully resolved are recorded as potential dependency edges, while parsing or fetching failures do not interrupt the main auditing workflow. This design improves analysis depth for composite attacks while maintaining robustness under incomplete external information.

3.5 Adaptive Iterative Budget Management

Deep vulnerability reasoning requires a balance between analytical depth and computational cost. A fixed iteration limit may lead to insufficient exploration for complex protocols or unnecessary computation for simple cases. The system therefore adopts an adaptive reasoning budget mechanism. Each vulnerability pattern is evaluated according to three characteristics: detection difficulty, reasoning complexity, and dependence on external knowledge. These factors are combined with protocol scale information to determine the required reasoning depth. Complex and high-value protocols receive additional analysis iterations, while simpler cases are constrained to fewer rounds. This adaptive strategy allows the system to allocate computational resources according to security complexity rather than relying on a predefined iteration limit.

3.6 System Implementation

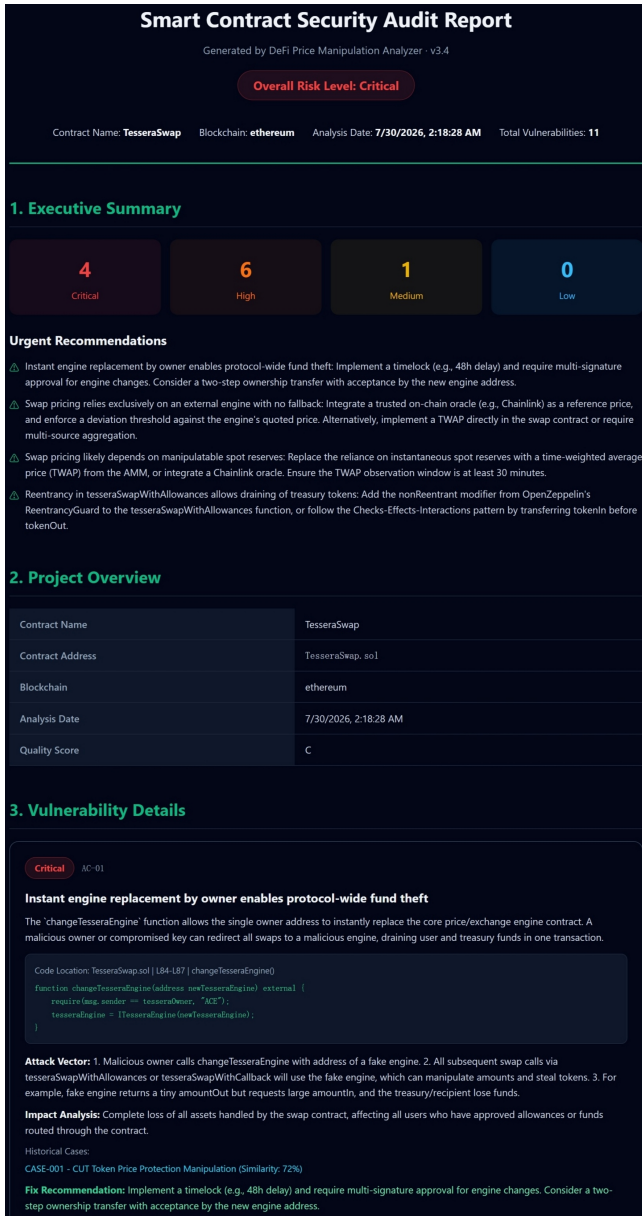
Building upon the technical architecture, the proposed framework is encapsulated within a web-based interactive platform comprising six core modules: Dashboard, Analyze, Cases, Patterns, History, and Settings. The primary workflow centers on the Analyze interface, which supports dual input modes: submitting on-chain addresses—with source code fetched via a cascading strategy ranging from verified API retrieval to bytecode decompilation—or uploading local source files directly. During execution, a real-time visualization panel tracks the seven-stage pipeline progress, providing immediate feedback on the audit status.

To enhance operational robustness and usability, the system incorporates several practical features. An automatic state preservation mechanism ensures that intermediate results are saved as a partial report if the analysis is interrupted, such as by an LLM API quota exhaustion, preventing data loss. A Batch Audit function allows for the simultaneous evaluation of multiple historical cases, while a Demo Mode facilitates rapid system testing with pre-loaded vulnerable contract samples. Additionally, the platform integrates a knowledge base of historical exploits and vulnerability patterns, enabling users to cross-reference findings through multi-dimensional filtering. Generated reports support bilingual output and are available for download in HTML, PDF, and JSON formats. The system interface is shown in Figure 2 and Figure 3. The source code of the system has been publicly released on GitHub¹.

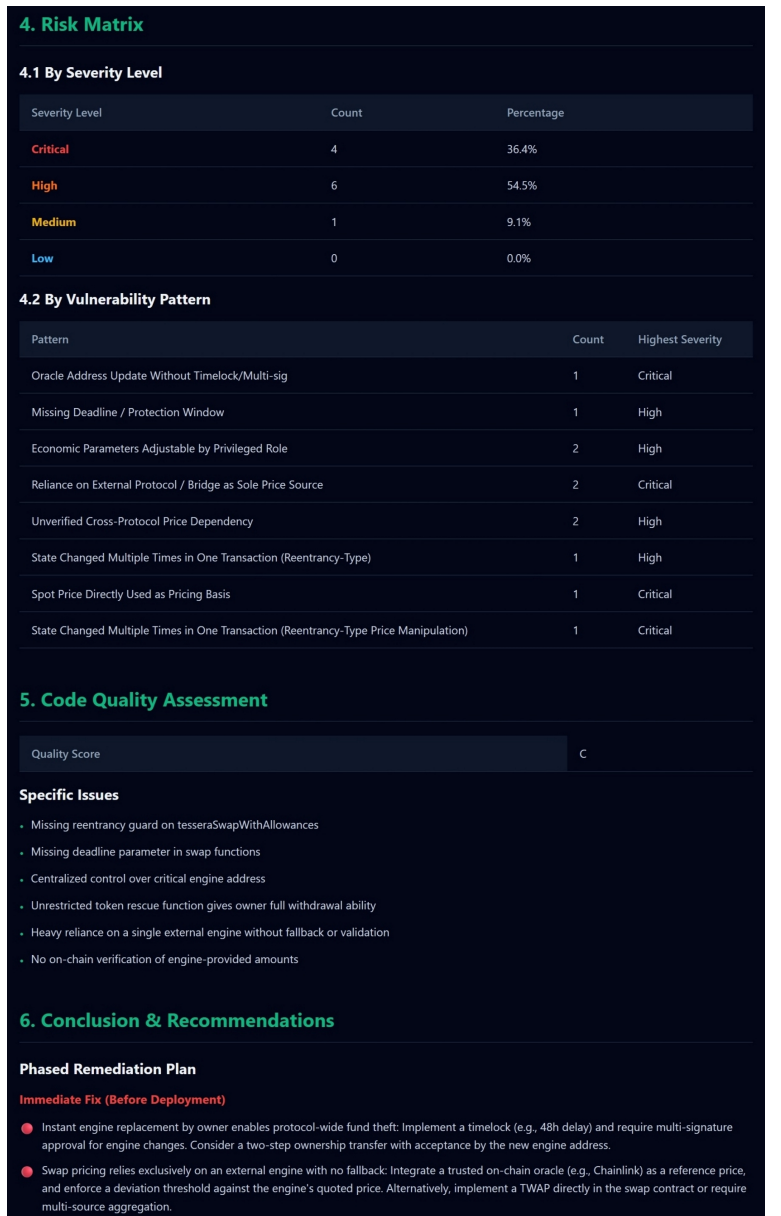


Figure 2: System Contract Analysis Interface

¹<https://github.com/bzhengak/Multi-Agent-DeFi-Price-Manipulation-Audit-System.git>



(a) Audit Report Partial Screenshot 1



(b) Audit Report Partial Screenshot 2

Figure 3: Audit Report Partial Screenshots

4 Evaluation

The evaluation investigates two primary questions: whether the proposed multi-agent framework can effectively identify DeFi price manipulation vulnerabilities, and whether the architectural design provides advantages over simpler auditing approaches. The dataset contains ten positive samples and ten negative samples. Positive cases are collected from reproducible DeFi exploit examples with manually verified root causes, covering 12 of the 21 vulnerability patterns from the proposed taxonomy. Negative cases are selected from professionally audited protocols without known price-manipulation incidents. Two baselines are considered: Raw-LLM, which uses the same LLM without agent coordination, context construction, or iterative reasoning, and Slither, a static analysis tool mapped to the supported vulnerability patterns. A key challenge in DeFi security evaluation is incomplete vulnerability labeling. Public exploit records usually identify the vulnerability responsible for an attack but do not necessarily enumerate all other valid security issues within the affected protocol. Therefore, the evaluation adopts a two-layer perspective on positive cases. The first layer focuses on root-cause detection, measuring whether the system identifies the vulnerability mechanism responsible for historical exploits. The second layer evaluates broader vulnerability coverage on fully annotated cases, preventing valid but historically unused findings from being incorrectly treated as false positives.

The evaluation uses four complementary metrics to measure different aspects of auditing capability. Root-Cause Recall measures whether the system successfully identifies the vulnerability pattern responsible for a historical exploit, indicating its ability to recover real attack causes. Positive-Case Pattern Recall evaluates the coverage of all confirmed vulnerabilities in fully annotated cases, quantifying the breadth of vulnerability discovery beyond the original ex-

exploit path. Positive-Case Pattern Precision measures the proportion of reported findings that correspond to valid vulnerabilities, reflecting the reliability of detected issues. Specificity evaluates the system’s ability to avoid reporting vulnerabilities on known-safe protocols, measuring its restraint against false alarms. Results are summarized in Table 4.

Table 4: Main evaluation results compared to baselines

Scope	Metric	Proposed System	Raw-LLM	Slither [†]
Positive-Case Root Cause [§]	Recall	92.9% (13/14)	50.0% (7/14)	0.0% (0/14)
Positive-Case Full Vulnerability	Recall	86.2% (50/58)	24.1% (14/58)	3.4% (2/58)
	Precision	87.7% (50/57)	70.0% (14/20)	50.0% (2/4)
Negative Case	Specificity [‡]	100% (10/10)	90.0% (9/10)	90.0% (9/10)

[†]Slither only covers 3/21 pattern types; 70% of contracts fail to compile.

[§]Root-Cause Precision omitted: $\frac{\text{root causes}}{\text{total detections}}$ penalizes detection of valid non-root-cause vulnerabilities.

[‡]Precision on Negatives.

Compared with Raw-LLM and Slither, the proposed framework achieves broader pattern coverage while maintaining high precision, suggesting that workflow decomposition and structured reasoning contribute significantly to audit reliability. The improvement is mainly attributed to the combination of protocol-aware context construction, iterative vulnerability analysis, cross-contract reasoning, and deterministic verification mechanisms.

The evaluation also highlights the difficulty of measuring DeFi security systems using incomplete labels. Some findings may not correspond directly to historical exploit causes but can still represent valid security concerns. Therefore, root-cause recall and broader vulnerability coverage should be interpreted together rather than relying on a single metric. The combination of multiple evaluation perspectives provides a more accurate assessment of automated DeFi auditing capability.

A root-cause analysis of the six false positives reported is shown in Table 5; half of them are semantic duplicates (same flaw triggering multiple pattern IDs), and the rest are genuine misclassifications of protection mechanisms.

Table 5: Breakdown of false positives in Layer 2 evaluation

Contract	FP Pattern	Reason
DLMCToken	OD-03	Duplicate with AC-02
DLMCToken	TO-03	Actual ReentrancyGuard present
OLPCToken	AC-01	'setSwapPair()' is an AC-02 parameter
TOPBPool	CL-01	Negligible rounding, no viable exploit path
TOPBPool	OD-01	Contract does not contain pricing logic
SQTokenStaking	OD-03	Duplicate with AC-02
SQTokenStaking	OD-02	Duplicate with OD-01

5 Limitations

The current system has several limitations. First, the vulnerability pattern library is manually maintained and does not automatically synchronize with emerging threat intelligence. As new DeFi attack mechanisms appear, additional patterns and knowledge updates are required to maintain coverage. Second, cross-contract tracing relies primarily on static analysis of contract relationships. Dynamic behaviors, such as proxy-based calls or runtime-generated addresses, may not always be fully resolved, potentially limiting dependency analysis in complex protocols. Third, although confidence calibration and deterministic filtering reduce unreliable findings, the system does not currently integrate formal verification techniques such as symbolic execution or taint analysis as an additional validation layer. Therefore, some semantic misclassification risks remain. Finally, cost estimation depends on external market information such as gas prices and asset values. While the estimation process is deterministic, it does not simulate complete on-chain execution, which limits accuracy for highly complex multi-step attacks.

6 Conclusion

This work presents a multi-agent framework for automated DeFi price manipulation security auditing. The system combines specialized agents, deterministic analysis, language-model-based reasoning, memory-assisted knowledge reuse, and confidence calibration into a structured auditing workflow. By following the principle of prioritizing deterministic computation over unrestricted model inference, the framework reduces hallucination risks while preserving the ability to reason about complex protocol interactions. The integration of cross-contract tracing and adaptive reasoning depth further improves the analysis of composite attacks that are difficult for traditional rule-based tools to capture.

Evaluation results demonstrate that the proposed architecture improves vulnerability discovery and pattern coverage compared with single-invocation LLM analysis and static analysis baselines, while maintaining reliable behavior on secure protocols. Although limitations remain in knowledge updating, dynamic dependency analysis, and automated verification, the framework provides a practical direction toward scalable and explainable DeFi security auditing.

References

- [1] Werner S, Perez D, Gudgeon L, et al. *SoK: Decentralized Finance*. Financial Cryptography (FC), 2021.
- [2] Mazon R, et al. *Beanstalk: A Case Study of DeFi Price Manipulation Attacks*. IEEE S&P, 2022.
- [3] Feist J, Grieco G, Groce A. *Slither: A Static Analysis Framework for Smart Contracts*. IEEE European Symposium on Security and Privacy (EuroS&P), 2019.
- [4] Das A, Gervais A, et al. *Securify: Practical Security Verification for Smart Contracts*. ACM CCS, 2019.
- [5] Wei J, Wang X, Schuurmans D, et al. *Chain-of-thought prompting elicits reasoning in large language models*. NeurIPS, 2022.
- [6] Pearce H, Ahmad B, et al. *Examining Zero-Shot Vulnerability Repair with Large Language Models*. IEEE S&P, 2022.
- [7] Daian P, Goldfeder S, et al. *Flash Boys 2.0: Frontrunning in Decentralized Exchanges, Transaction Ordering, and Consensus*. IEEE S&P, 2020.
- [8] Liu Y, Chen X, et al. *Multi-Agent Collaboration for Explainable Smart Contract Auditing*. ACM CCS, 2023.
- [9] Smart Contract Weakness Classification (SWC) Registry. <https://swcregistry.io/>, 2020.
- [10] Wu S, et al. *DeFiRanger: Detecting Price Manipulation Attacks on DeFi Applications*. arXiv, 2021.
- [11] Li W, Liu Z, et al. *Comprehensive Review of Smart Contract and DeFi Security*. Expert Systems with Applications, 2025.
- [12] Carpentier-Desjardins C, et al. *Mapping the DeFi Crime Landscape*. arXiv, 2023.
- [13] Jiang S, et al. *DeFi Security: A Survey*. High-Confidence Computing, 2026.
- [14] OWASP Smart Contract Top 10, 2026.
- [15] Zhang L, et al. *Confidence Calibration for Automated Security Auditing*. NeurIPS, 2024.
- [16] Reyes A, et al. *Advanced Static Analysis for DeFi Vulnerability Detection*. IEEE S&P, 2023.
- [17] Cao Y. *In-depth Analysis of Common Patterns of DeFi Economic Attacks*. HashKey Research, 2021.